

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA0301

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

CODING CHALLENGE

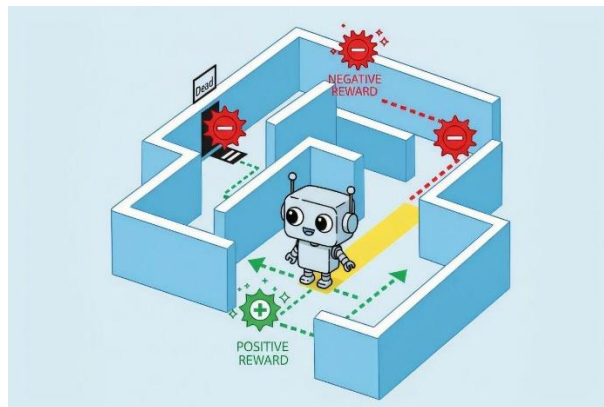
Duration: 2hrs

1. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance.

(5 marks)

Introduction

Reinforcement Learning (RL) is a machine learning technique that enables robots to learn optimal actions by interacting with their environment and receiving rewards or penalties based on their performance. Unlike traditional programming, RL allows robots to improve their behavior through continuous experience rather than following predefined instructions. This makes RL highly suitable for applications such as autonomous vehicles, industrial robots, warehouse automation, drone navigation, and healthcare robotics. However, implementing RL in real-world robotics is much more challenging than in simulated environments. Physical robots operate in dynamic and unpredictable conditions where safety, hardware limitations, and environmental uncertainty must be considered. In addition, collecting training data from real robots is expensive and time-consuming, making efficient learning essential. Therefore, overcoming these challenges is crucial for developing reliable and intelligent robotic systems.



Challenges of Applying Reinforcement Learning in Real-World Robotics

1. Safety Challenges

One of the biggest challenges in applying RL to robotics is ensuring the safety of both the robot and its surroundings. During the learning process, an RL agent explores different actions, including actions that may be unsafe. Such actions can cause collisions, equipment damage, or even injuries to humans working nearby. Therefore, robots require safe exploration strategies and safety constraints to prevent accidents while learning.

Key Points

- Exploration may lead to dangerous actions.
- Unsafe actions can damage hardware.
- Human safety must be ensured.
- Safe RL algorithms reduce operational risks.
- Safety constraints are incorporated during training.

2. Sample Efficiency

Most Reinforcement Learning algorithms require a large number of interactions with the environment before learning an optimal policy. In robotics, every interaction involves physical movement, power consumption, and execution time, making the learning process costly. Unlike simulations, real robots cannot perform millions of training episodes quickly. Therefore, improving sample efficiency is essential to reduce both training time and operational costs.

Key Points

- RL requires thousands or millions of training experiences.
- Collecting real-world data is expensive.
- Training robots takes significant time.
- Efficient algorithms reduce data requirements.
- Simulation-based pre-training improves efficiency.

3. Environment Variability

Real-world environments are constantly changing, making robotic learning more difficult. Factors such as lighting conditions, moving obstacles, uneven surfaces, weather changes, and sensor noise can significantly affect the robot's performance. An RL agent trained in one environment may fail when deployed in a different environment. Therefore, robots must learn robust policies that can adapt to varying conditions.

Key Points

- Real environments are unpredictable.
- Sensor noise affects decision-making.
- Dynamic obstacles increase complexity.
- Robots must generalize to unseen situations.
- Adaptive learning improves robustness.

4. Simulation-to-Real Gap

Many RL models are initially trained in simulation because simulations are faster, cheaper, and safer than real-world training. However, simulated environments cannot perfectly replicate real-world conditions. Differences in physics, friction, sensor accuracy, and environmental factors often cause a trained policy to perform poorly when transferred to an actual robot. This difference is known as the **Simulation-to-Real (Sim-to-Real) Gap**.

Key Points

- Simulation differs from real-world conditions.
- Physical properties may not match perfectly.
- Policies may fail after deployment.
- Domain randomization helps reduce the gap.
- Fine-tuning on real robots improves performance.

5. Computational Complexity

Modern RL algorithms, especially Deep Reinforcement Learning methods, require significant computational resources. Training involves processing large datasets and repeatedly updating neural networks, which demands high-performance CPUs or GPUs. Robots with limited onboard computing power may struggle to execute these algorithms in real time, leading to delays in decision-making.

Key Points

- Deep RL requires powerful hardware.
- Training consumes significant computation time.
- Memory requirements are high.
- Real-time decision-making becomes challenging.
- Cloud or edge computing can assist processing.

6. Reward Function Design

The reward function guides the robot toward the desired behavior. Designing an appropriate reward function is difficult because poorly designed rewards may encourage unintended actions or inefficient learning. The reward should accurately reflect the task objective while avoiding behaviors that exploit loopholes in the reward structure.

Key Points

- Reward design directly affects learning.
- Incorrect rewards lead to poor performance.
- Rewards should encourage desired behavior.
- Sparse rewards slow learning.
- Well-designed rewards improve convergence.

7. Exploration vs. Exploitation

A robot must balance **exploration** (trying new actions) and **exploitation** (using learned knowledge). Excessive exploration may result in unsafe or inefficient behavior, whereas excessive exploitation may prevent the robot from discovering better solutions. Maintaining the right balance is essential for successful learning.

Key Points

- Exploration discovers new strategies.
- Exploitation uses learned knowledge.
- Too much exploration increases risk.
- Too much exploitation limits improvement.
- Balanced strategies improve learning efficiency.

Impact on Agent Performance

The above challenges directly influence the overall performance of an RL agent in robotics. Safety constraints may limit exploration but improve reliability. Poor sample efficiency increases training time and operational costs. Environment variability reduces the agent's ability to generalize to new situations, while computational limitations slow decision-making. Additionally, an improperly designed reward function may lead to undesirable behaviors. Addressing these challenges results in robots that are more adaptive, efficient, and capable of performing reliably in complex real-world environments.

Applications of RL in Robotics

- Autonomous self-driving vehicles
- Warehouse automation robots
- Industrial robotic arms
- Medical and surgical robots
- Agricultural field robots
- Drone navigation and surveillance
- Service robots in homes and hospitals

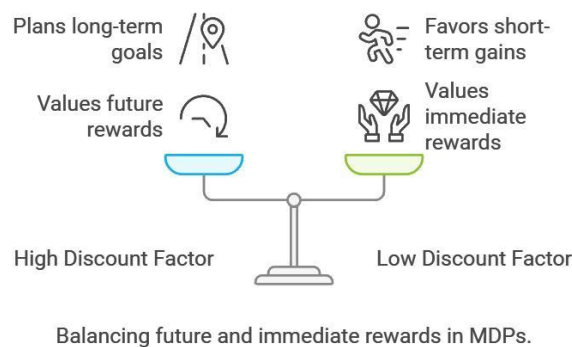
Conclusion

Reinforcement Learning offers a powerful approach for enabling robots to learn complex tasks without explicit programming. However, applying RL in real-world robotics presents significant challenges, including safety, sample efficiency, environment variability, simulation-to-real transfer, computational complexity, reward design, and balancing exploration with exploitation. By developing safer learning algorithms, improving sample efficiency, and creating robust policies that adapt to changing environments, researchers can make RL-based robotic systems more reliable and practical for real-world applications.

2. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks. (5 marks)

Introduction

The **discount factor** (γ) is one of the most important parameters in Reinforcement Learning (RL) because it determines how much importance an agent gives to future rewards compared to immediate rewards. It influences the learning strategy of the agent by controlling whether decisions are made for short-term gains or long-term benefits. The value of γ ranges from **0 to 1**, where a smaller value focuses on immediate rewards, while a larger value encourages the agent to consider future outcomes. Selecting an appropriate discount factor is essential because it directly affects the quality of the learned policy and the agent's ability to maximize cumulative rewards. In episodic tasks, where interactions end after reaching a terminal state, the discount factor helps the agent optimize the total reward throughout the entire episode.



Discount Factor (γ)

The discount factor is represented by the symbol γ (**gamma**) and is mathematically expressed in the return equation:

$$G = R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 + \dots$$

Where:

- G = Total discounted return
- $R_1, R_2, R_3, \dots$ = Rewards received at different time steps
- γ ($0 \leq \gamma \leq 1$) = Discount factor

The value of γ determines how much future rewards contribute to the total return.

Influence on Short-Term Decision Making (Low γ)

When the discount factor is **close to 0**, the agent gives greater importance to immediate rewards while ignoring rewards that may occur later. This makes the learning process faster because the agent focuses only on nearby outcomes. However, the agent may fail to discover strategies that provide higher rewards in the future.

Explanation

For example, consider a warehouse robot that has two possible paths:

- **Path A:** Collects a small reward immediately.
- **Path B:** Travels a longer distance but receives a much larger reward later.

If $\gamma = 0.2$, the robot prefers **Path A** because immediate rewards are valued more than future rewards.

Key Points

- Focuses on immediate rewards.
- Ignores distant future rewards.
- Faster convergence.
- Suitable for short-duration tasks.
- May miss better long-term solutions.

Influence on Long-Term Decision Making (High γ)

When the discount factor is **close to 1**, the agent values future rewards almost as much as immediate rewards. This encourages the agent to plan ahead and learn strategies that maximize cumulative rewards over time. Although learning may take longer, the resulting policy is often more optimal.

Explanation

Consider a self-driving car approaching a traffic signal:

- It may slow down now (small immediate cost).
- By doing so, it avoids accidents and reaches the destination safely (greater future reward).

With $\gamma = 0.95$, the agent selects the safer long-term strategy.

Key Points

- Gives importance to future rewards.
- Encourages long-term planning.
- Learns better strategies.
- Produces higher cumulative rewards.
- Requires more training time.

Comparison of Low and High Discount Factors

Low γ (Near 0)	High γ (Near 1)
Focuses on immediate rewards	Focuses on future rewards
Short-term decision making	Long-term decision making
Faster learning	Slower learning
Suitable for simple tasks	Suitable for complex planning
May ignore better future rewards	Maximizes cumulative rewards

Role of Discount Factor in Episodic Tasks

An **episodic task** is a task that has a clear beginning and ending. Once the goal is reached or the episode ends, the environment resets for a new episode. Examples include maze navigation, robot path planning, chess games, and autonomous driving simulations.

In episodic tasks, the discount factor helps the agent evaluate not only the immediate reward but also the rewards that can be accumulated before the episode terminates. A higher discount factor enables the agent to complete the entire task efficiently by considering future consequences of current actions.

Examples of Episodic Tasks

- Robot navigating a maze.
- Autonomous drone delivering a package.
- Chess game.
- Robot arm assembling a product.
- Warehouse robot picking and placing objects.

Key Points

- Maximizes cumulative reward during an episode.
- Helps learn complete action sequences.
- Improves planning toward the terminal state.
- Balances immediate and delayed rewards.
- Enhances policy optimization.

Applications of Discount Factor

The discount factor is widely used in many Reinforcement Learning applications, including:

- Autonomous vehicles
- Robotics and industrial automation
- Game-playing AI (Chess, Go, Atari)
- Healthcare treatment planning
- Financial trading systems
- Recommendation systems
- Smart traffic signal control

Advantages of Selecting an Appropriate Discount Factor

- Improves learning efficiency.
- Produces better decision-making.
- Balances immediate and future rewards.
- Helps achieve optimal policies.
- Increases overall agent performance.

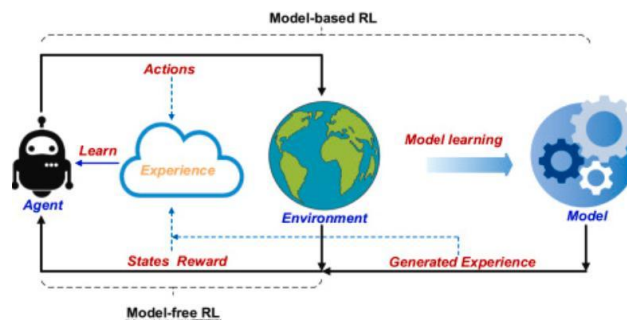
Conclusion

The discount factor (γ) plays a crucial role in Reinforcement Learning by controlling the balance between immediate and future rewards. A **low** γ encourages short-term decision-making by emphasizing immediate gains, whereas a **high** γ promotes long-term planning and maximizes cumulative rewards. In episodic tasks, the discount factor enables the agent to consider the entire sequence of actions before reaching the terminal state, leading to more effective and intelligent behavior. Therefore, selecting an appropriate value of γ is essential for developing efficient RL agents capable of solving complex real-world problems.

3. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations. (5 marks)

Introduction

Reinforcement Learning (RL) algorithms are broadly classified into **Model-Free RL** and **Model-Based RL** based on whether they use a model of the environment during learning. Model-Free RL learns directly through trial-and-error interactions without having prior knowledge of the environment, whereas Model-Based RL first learns or utilizes an environment model to predict future states and rewards before making decisions. Both approaches have become widely used in robotics, autonomous systems, gaming, healthcare, and industrial automation. In dynamic environments, where conditions change frequently, selecting the appropriate RL approach is essential for achieving efficient learning and reliable decision-making. Each method has unique strengths and limitations that influence its performance depending on the complexity of the environment and the availability of computational resources.



Model-Free Reinforcement Learning

Model-Free Reinforcement Learning enables the agent to learn an optimal policy directly through interactions with the environment. It does not require any prior knowledge or mathematical model of the environment's dynamics. Instead, the agent continuously explores different actions, receives rewards or penalties, and gradually improves its decision-making policy over time.

Examples

- Q-Learning
- SARSA
- Deep Q-Network (DQN)

Advantages

- Does not require an environment model.
- Easy to implement.
- Works well in unknown environments.
- Flexible for complex real-world problems.
- Suitable when the environment changes frequently.

Limitations

- Requires a large amount of training data.
- Learning process is slow.
- High computational cost.
- Low sample efficiency.
- Exploration can be time-consuming.

Model-Based Reinforcement Learning

Model-Based Reinforcement Learning uses a mathematical or learned model of the environment to predict future states, rewards, and outcomes before taking actions. Instead of relying solely on trial-and-error, the agent performs planning using the environment model, allowing it to make more informed decisions with fewer interactions.

Examples

- Dyna-Q
- Monte Carlo Tree Search (MCTS)
- MuZero

Advantages

- Faster learning.
- Better sample efficiency.
- Requires fewer real-world interactions.
- Supports planning before action execution.
- Suitable for expensive or safety-critical environments.

Limitations

- Building an accurate model is difficult.
- Model errors reduce learning performance.
- Higher implementation complexity.
- Computational overhead for planning.
- Less effective when the environment changes rapidly.

Effectiveness in Dynamic Environments

Dynamic environments are environments where conditions continuously change due to moving objects, changing weather, sensor noise, or unpredictable events. Examples include autonomous driving, drone navigation, warehouse robotics, and traffic management.

Model-Free RL in Dynamic Environments

Model-Free RL adapts by continuously learning from new experiences. Since it does not depend on a fixed environment model, it can adjust its policy when the environment changes. However, adaptation may require many additional interactions, making learning slower.

Key Points

- Adapts through continuous interaction.
- Performs well in unknown environments.
- Handles unpredictable situations.

- Requires more exploration.
- Slower adaptation speed.

Model-Based RL in Dynamic Environments

Model-Based RL performs efficiently when the environment model accurately represents the current situation. It enables faster decision-making by predicting future outcomes. However, if the environment changes significantly, the model becomes inaccurate and must be updated, increasing computational complexity.

Key Points

- Faster planning and decision-making.
- More sample efficient.
- Depends on model accuracy.
- Model updates are required in changing environments.
- Better for structured environments.

Comparison Between Model-Free and Model-Based RL

Model-Free RL	Model-Based RL
Learns through trial and error	Learns using an environment model
No model required	Requires a model
Slower learning	Faster learning
High data requirement	Low data requirement
Lower sample efficiency	Higher sample efficiency
Easy to implement	More complex to implement

Applications

Model-Free RL

- Autonomous robots
- Video game AI
- Recommendation systems
- Smart traffic control
- Robot navigation

Model-Based RL

- Self-driving vehicles
- Industrial robotics
- Drone path planning
- Medical treatment planning
- Manufacturing automation

Strengths and Limitations Summary

Strengths of Model-Free RL

- Simple implementation.
- No prior knowledge required.
- Flexible and adaptable.
- Suitable for complex unknown environments.

Limitations of Model-Free RL

- Slow convergence.
- Requires large datasets.
- High computational cost.
- Less sample efficient.

Strengths of Model-Based RL

- Fast convergence.
- Efficient use of training data.
- Supports planning.
- Better performance in structured environments.

Limitations of Model-Based RL

- Difficult to build accurate models.
- Sensitive to modeling errors.
- High planning complexity.
- Less adaptable if the model becomes outdated.

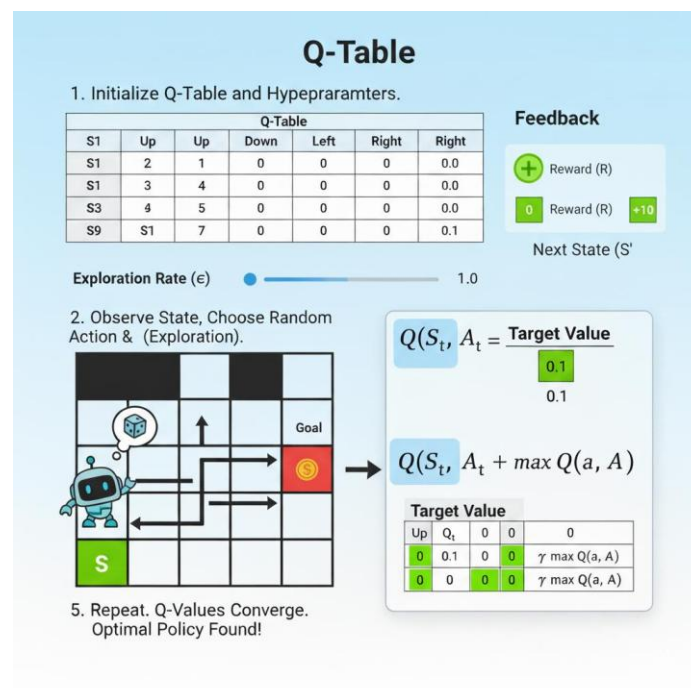
Conclusion

Both Model-Free and Model-Based Reinforcement Learning have significant advantages in solving sequential decision-making problems. Model-Free RL is highly flexible and suitable for unknown or highly dynamic environments because it learns directly from experience without relying on an environment model. However, it requires large amounts of data and longer training time. In contrast, Model-Based RL achieves faster learning and greater sample efficiency by using an environment model for planning, but its performance depends heavily on the accuracy of that model. Therefore, the choice between these approaches depends on the application's requirements, the availability of an accurate environment model, and the level of environmental uncertainty.

4. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior. (5 marks)

Introduction

Q-Learning is one of the most popular **model-free** and **off-policy Reinforcement Learning (RL)** algorithms used to determine the optimal action for an agent in a given environment. It enables an agent to learn the best policy through trial-and-error interactions without requiring prior knowledge of the environment. During training, the agent repeatedly observes the current state, performs an action, receives a reward, and updates its knowledge using the Q-value update equation. These Q-values represent the expected future reward for taking a particular action in a specific state. As the learning process continues, the Q-values gradually become more accurate, allowing the agent to make optimal decisions. Eventually, the algorithm converges to the optimal policy after sufficient exploration and repeated updates.



Q-Learning

Q-Learning learns the **action-value function (Q-function)**, which estimates the expected cumulative reward of taking an action in a given state and then following the optimal policy.

The Q-value is updated using the following equation:

$$Q(s', a) = Q(s', a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s', a)]$$

Where:

- **Q(s,a)** = Current Q-value
- **α (Alpha)** = Learning rate

- γ (**Gamma**) = Discount factor
- r = Immediate reward
- s = Current state
- a = Current action
- s' = Next state
- $\max Q(s',a')$ = Maximum expected future reward

Working Process of Q-Learning

The Q-Learning algorithm follows a sequence of steps to update its action-value function during training.

Steps

1. Observe the current state.
2. Select an action using an exploration strategy.
3. Execute the selected action.
4. Receive an immediate reward.
5. Move to the next state.
6. Update the Q-value using the Q-Learning equation.
7. Repeat the process until the goal is achieved.

Key Points

- Learns through trial and error.
- Updates Q-values after every interaction.
- Does not require an environment model.
- Learns the optimal policy over time.
- Suitable for unknown environments.

Example of Q-Learning Update

Consider a robot navigating through a warehouse to reach a delivery point.

Suppose the following values are given:

- Current Q-value = **10**
- Learning Rate (α) = **0.5**
- Discount Factor (γ) = **0.9**
- Immediate Reward (r) = **8**
- Maximum Future Q-value = **12**

Step 1: Calculate the Target Value

Target Reward

$$= 8 + (0.9 \times 12)$$

$$= 18.8$$

Step 2: Update the Q-value

New Q-value

$$= 10 + 0.5 (18.8 - 10)$$

$$= 10 + 4.4$$

$$= 14.4$$

Therefore,

Old Q-value = 10

Updated Q-value = 14.4

This updated Q-value indicates that taking the selected action from the current state is more beneficial than previously estimated.

Action-Value Function Update

The action-value function stores the estimated reward for every state-action pair.

Example Q-table:

State	Action	Old Q-value	Updated Q-value
S1	Move Right	10	14.4
S2	Move Left	7	8.2
S3	Move Forward	5	6.8

the Q-table is continuously updated during training until it accurately represents the best actions.

Convergence Behavior

Convergence refers to the process by which Q-values gradually stabilize and approach their optimal values after repeated learning experiences.

Initially, the Q-values are random or inaccurate. As the agent interacts with the environment repeatedly, it updates the Q-values using newly received rewards. After sufficient exploration and repeated updates, the changes become smaller, and the Q-values eventually stabilize. At this stage, the learned policy is considered optimal.

Key Points

- Q-values improve after each interaction.
- Learning errors gradually decrease.

- The policy becomes more accurate over time.
- Convergence requires sufficient exploration.
- Stable Q-values indicate optimal learning.

Advantages of Q-Learning

- Simple and easy to implement.
- Does not require prior knowledge of the environment.
- Learns the optimal policy independently.
- Works well for discrete state-action spaces.
- Suitable for robotics, gaming, and navigation problems.

Limitations of Q-Learning

- Slow learning in large environments.
- Requires extensive exploration.
- Large Q-tables consume more memory.
- Less efficient for continuous state spaces.
- Performance depends on proper parameter selection.

Applications of Q-Learning

- Robot navigation
- Autonomous vehicles
- Game-playing AI
- Warehouse automation
- Smart traffic signal control
- Recommendation systems
- Industrial process optimization

Conclusion

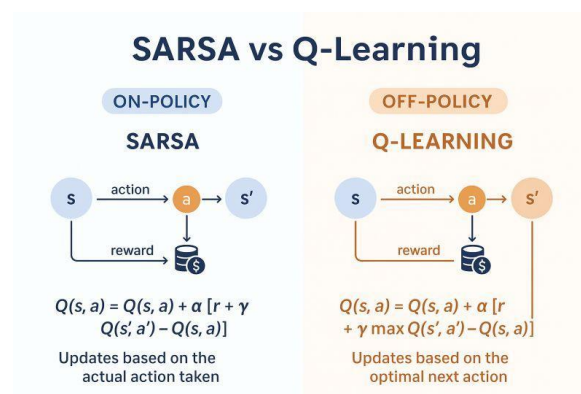
Q-Learning is a powerful Reinforcement Learning algorithm that enables an agent to learn the optimal action-value function through continuous interaction with its environment. By repeatedly updating Q-values using rewards and future estimates, the agent gradually improves its decision-making ability. As training progresses, the Q-values converge toward optimal values, resulting in a stable and efficient policy. Due to its simplicity and effectiveness, Q-Learning is widely used in robotics, autonomous systems, game AI, and many other real-world decision-making applications.

5. Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications.

(5 marks)

Introduction

Reinforcement Learning (RL) algorithms are classified into **On-Policy** and **Off-Policy** learning methods based on how the agent learns from its experiences. These methods determine whether the agent learns using the same policy that generates its actions or from a different policy. Both approaches aim to maximize cumulative rewards, but they differ in the way they collect and utilize experience during training. Understanding the differences between On-Policy and Off-Policy learning is essential for selecting the most suitable algorithm for a particular application. The choice depends on factors such as learning speed, stability, sample efficiency, and the complexity of the environment. These learning methods are widely used in robotics, autonomous systems, game playing, finance, healthcare, and recommendation systems.



On-Policy Learning

On-Policy learning is a Reinforcement Learning approach in which the agent **learns using the same policy that it follows to select actions**. The policy used for exploration and the policy being improved are identical. As the agent gains new experiences, it immediately updates the same policy, making learning stable and consistent.

Example

- **SARSA (State–Action–Reward–State–Action)**

Working Process

1. Observe the current state.
2. Select an action using the current policy.
3. Execute the action.
4. Receive a reward.
5. Observe the next state.
6. Select the next action using the same policy.
7. Update the value function and repeat.

Advantages

- Stable learning process.
- Safer exploration.
- Consistent policy updates.
- Better suited for real-world robotics.
- Lower risk of unsafe actions.

Limitations

- Slower learning speed.
- Lower sample efficiency.
- Requires continuous interaction.
- May converge more slowly.
- Less effective in large environments.

Off-Policy Learning

Off-Policy learning allows the agent to **learn an optimal policy while following a different behavior policy**. The experiences used for learning do not necessarily come from the policy being optimized. This enables the agent to reuse previously collected experiences, improving learning efficiency.

Examples

- **Q-Learning**
- **Deep Q-Network (DQN)**

Working Process

1. Observe the current state.
2. Select an action using a behavior policy.
3. Execute the action.
4. Receive a reward.
5. Move to the next state.
6. Update the optimal policy using the maximum future reward.
7. Repeat until convergence.

Advantages

- Faster learning.
- High sample efficiency.
- Can reuse stored experiences.
- Learns the optimal policy effectively.
- Suitable for complex environments.

Limitations

- Less stable than On-Policy methods.
- Sensitive to parameter selection.
- Risk of overestimating action values.
- More difficult to implement.
- Requires careful tuning.

Comparison Between On-Policy and Off-Policy Learning

On-Policy Learning	Off-Policy Learning
Learns using the current policy	Learns using a different behavior policy
Example: SARSA	Example: Q-Learning, DQN
Stable learning	Faster learning
Lower sample efficiency	Higher sample efficiency
Safer exploration	Better data reuse
Slower convergence	Faster convergence
Suitable for safety-critical tasks	Suitable for data-intensive applications

Practical Applications

On-Policy Learning

On-Policy methods are preferred in applications where **safety and stable behavior** are important.

Applications

- Autonomous mobile robots
- Self-driving vehicles
- Healthcare robotics
- Industrial automation
- Drone navigation

Off-Policy Learning

Off-Policy methods are widely used in applications that require **efficient learning from large datasets** and the ability to reuse past experiences.

Applications

- Game-playing AI
- Warehouse automation
- Recommendation systems
- Financial trading
- Smart traffic management
- Robotics simulation

Advantages and Limitations Summary

Advantages of On-Policy Learning

- Stable convergence.
- Safer decision-making.
- Easy policy interpretation.
- Reliable learning behavior.
- Suitable for real-world control systems.

Limitations of On-Policy Learning

- Slow convergence.
- Requires more interactions.
- Less sample efficient.
- Limited data reuse.
- Higher training time.

Advantages of Off-Policy Learning

- Faster convergence.
- Better sample efficiency.
- Can learn from historical data.
- More flexible.
- Suitable for large-scale problems.

Limitations of Off-Policy Learning

- Learning may become unstable.
- Sensitive to hyperparameters.
- Overestimation of Q-values is possible.
- Higher implementation complexity.
- Requires careful exploration strategies.

Conclusion

On-Policy and Off-Policy learning are two fundamental approaches in Reinforcement Learning, each offering unique benefits depending on the application. On-Policy learning updates the same policy used to generate actions, resulting in stable and safer learning, making it suitable for robotics and safety-critical systems. Off-Policy learning, on the other hand, learns an optimal policy from experiences generated by another policy, providing faster convergence and greater sample efficiency. While On-Policy methods prioritize stability and safety, Off-Policy methods emphasize learning efficiency and optimal performance. Therefore, selecting the appropriate learning method depends on the task requirements, environmental complexity, available training data, and computational resources.

Code of Conduct:

*I Malli Chandana(192472250) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Malli Chandana

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

